

CFLOWNETS: 使用生成流网络进行连续控制

Yinchuan Li¹, Shuang Luo^{2*}, Haozhi Wang³, Jianye Hao^{1,3}¹Huawei Noah's Ark Lab, Beijing, China²Zhejiang University, Huangzhou, China³Tianjin University, Tianjin, China

{liyinchuan, haojianye}@huawei.com, luoshuang@zju.edu.cn

wanghaozhi@tju.edu.cn

摘要

作为一种新兴技术,生成流网络(GFlowNets)可以作为探索性控制任务中强化学习的一种替代方法。GFlowNet 旨在生成与终止状态上的奖励成比例的分布,并以主动学习的方式采样不同的候选方案。GFlowNets 需要形成一个有向无环图,并通过遍历轨迹中每个节点的流入和流出计算流匹配损失。目前还没有实验得出结论,表明 GFlowNets 可以用于处理连续任务。在本文中,我们提出了生成连续流网络(CFlowNets),它可以应用于连续控制任务。首先,我们介绍了 CFlowNets 的理论公式。然后,提出了 CFlowNets 的训练框架,包括动作选择过程、流近似算法以及连续流匹配损失函数。接着,我们从理论上证明了流近似的误差界限。随着流样本数量的增加,误差迅速减小。最后,在连续控制任务上的实验结果表明,与许多强化学习方法相比,CFlowNets 具有性能优势,尤其是在探索能力方面。

1 引言

作为一种新兴技术,生成流网络(GFlowNets) (Bengio 等, 2021a;b) 可以弥补强化学习 (Kaelbling 等, 1996; Sutton & Barto, 2018) 在探索性任务上的不足。具体来说,基于贝尔曼方程 (Sutton & Barto, 2018),强化学习通常被训练以最大化未来奖励的期望;因此,所学策略更倾向于采样具有更高奖励的动作序列。相比之下,GFlowNets 的训练目标是定义一个与终止状态上的奖励成比例的分布,即最终状态的父状态,而不是生成单个高奖励动作序列 (Bengio 等, 2021a)。这更像是在主动学习设置中采样不同的候选方案 (Bengio 等, 2021b),因此更适合于探索任务。

GFlowNets 将轨迹的状态转换构造成一个有向无环图 (DAG) 结构。图结构中的每个节点对应不同的状态,而动作则对应不同状态之间的转换,即连接图中不同节点的边。对于离散任务,该图结构中的节点数量有限,每条边只能对应一种离散动作。然而,在实际环境中,许多任务的状态和动作空间是连续的,例如四足动物的步态 (Kohl & Stone, 2004)、自动驾驶 (Kiran 等, 2021; Shalev-Shwartz 等, 2016; Pan 等, 2017) 或灵巧的手内操作 (Andrychowicz 等, 2020)。此外,这些环境对应的奖励分布可能是多模态的,需要更多的多样性探索。这些环境的需求与 GFlowNets 的优势紧密匹配。(Bengio 等, 2021b) 提出了通过用积分替换连续变量的求和来适应 GFlowNets 处理连续任务的想法,* 通讯作者: 罗双。本研究在华为诺亚方舟实验室进行期间完成,罗双和王浩之为成员。

他们建议使用可积密度和细致平衡 (DB) 或轨迹平衡

(TB) Malkin 等 (2022) 提出的准则用于获得可处理的训练目标, 从而可以避免某些积分运算。然而, 这一想法尚未经过实验验证。

本文提出了生成式连续流网络, 简称 CFlowNets, 用于连续控制任务, 生成与连续奖励函数成比例的策略。将 GFlowNets 应用于连续控制任务极具挑战性。在生成流网络中, 转移概率定义为动作流与状态流之比。对于离散的状态和动作空间, 我们可以形成一个有向无环图, 并通过遍历节点的流入和流出流量来计算状态流。相反, 在连续任务中, 遍历所有状态-动作对及其对应的奖励是不可能的。为了解决这一问题, 我们使用重要抽样来近似流匹配约束中的流入和流出积分, 其中我们使用深度神经网络来预测采样轨迹中每个状态的父节点。本文的主要贡献总结如下:

主要贡献: 1) 我们扩展了先前 GFlowNets 的理论公式和流匹配定理至连续场景。基于此, 提出了用于训练 CFlowNets 的损失函数; 2) 我们提出了一种有效的方法来采样概率近似于流网络输出的动作, 并提出了一种流采样方法来逼近连续流入和流出, 这使得我们可以构建一个连续流匹配损失; 3) 我们理论上分析了采样流与流入/流出之间的误差界限, 随着流样本数量的增加, 尾部变得微不足道; 4) 我们基于连续控制任务进行了实验, 证明了 CFlowNets 可以超越当前最先进的 RL 算法, 尤其是在探索能力方面。据我们所知, 我们的工作首次实证展示了流网络在连续控制任务上的有效性。代码可在 <http://gitee.com/mindspore/models/tree/master/research/gflownets/cflownets> 获取

2 预备知识

2.1 马尔可夫决策过程

一个随机的、离散时间的和顺序决策任务可以用马尔可夫决策过程 (MDP) 来描述, 其规范公式由元组表示:

$$M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle. \quad (1)$$

在这个过程中, $s \in \mathcal{S}$ 表示环境的状态空间。在每个时间步, 智能体接收一个状态 s 并在动作空间 $\mathcal{A}$ 上选择一个动作 a 。这导致根据状态转移函数 $P(s'|s, a) : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ 转移到下一个状态 s' 。然后, 智能体根据奖励函数 $R(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 获取奖励 r 。随机策略 π 将每个状态映射到动作分布 $\pi(\cdot|s)$, 并给出在状态 s 中选择动作 a 的概率 $\pi(a|s)$ 。智能体通过执行策略 π 并获得合法轨迹 $\{(s_t, a_t, r_t, s_{t+1})\}_{t=1}^n$ 与环境交互, 其中 n 是轨迹长度。智能体的目标是最大化折扣回报 $\mathbb{E}_{s_0:n, a_0:n} [\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi]$, 其中 $\mathbb{E}$ 是轨迹分布的期望, 而 $\gamma \in [0, 1)$ 是折扣因子。

2.2 生成流网络

GFlowNet 将 MDP 视为流网络。定义 $s' = T(s, a)$ 和 $F(s)$ 为节点的转移和通过 s 的总流。定义边缘/动作流 $F(s, a) = F(s \rightarrow s')$ 为通过边缘 $s \rightarrow s'$ 的流。普通 GFlowNets 的训练过程需要通过节点 (状态) 汇总父节点和子节点的流, 这依赖于离散状态空间和离散动作空间。该框架通过以下流一致性方程进行优化: ∇

$$F(s, a) = R(s') + \sum_{a' \in \mathcal{A}(s')} F(s', a'), \quad (2)$$

这意味着对于任意节点 s , 流入的流量等于流出的流量, 即该节点的总流量。
 $F(s)$ s .

3 连续流网络：理论表述

考虑一个具有元组 $(\mathcal{S}, \mathcal{A})$ 的连续任务，其中 $\mathcal{S}$ 表示连续状态空间， $\mathcal{A}$ 表示连续动作空间。定义此连续任务中的轨迹 $\tau = (s_1, \dots, s_n)$ 为从 $\mathcal{S}$ 中采样的元素序列，使得每次转移 $a_t : s_t \rightarrow s_{t+1} \in \mathcal{A}$ 。进一步地，我们将无环轨迹 $\tau = (s_1, \dots, s_n)$ 定义为满足无环约束的轨迹：

$\forall s_m \in \tau, s_k \in \tau, m \neq k$ ，我们有 $s_m \neq s_k$ 。记 s_0 和 s_f 分别为与连续任务 $(\mathcal{S}, \mathcal{A})$ 相关的初始状态和最终状态，我们将完整的轨迹定义为从 s_0 开始并在 s_f 结束的任何采样的无环轨迹 $(\mathcal{S}, \mathcal{A})$ 。相应地，进入最终状态的转移 $s \rightarrow s_f$ 被定义为终止转移，而 $F(s \rightarrow s_f)$ 则是一个终止流。

轨迹流量 $F(\tau) : \tau \mapsto \mathbb{R}^+$ 被定义为在完整轨迹集合上定义的任何非负函数 τ 。对于每个轨迹 τ ，相关联的流量 $F(\tau)$ 包含共享相同路径的粒子数量 (Bengio 等人, 2021b) τ 。此外，元组 $(\mathcal{S}, \mathcal{A}, F)$ 被称为连续流网络。令 $T(s, a) = s'$ 表示一个可以将状态从 s 转移到达成 s' 的动作 a 。然后我们做出以下假设。

假设 1。 假设连续任务 $(\mathcal{S}, \mathcal{A})$ 是一个“无环”的任务，这意味着任意采样的轨迹 τ 都是无环的，即

$$s_i \neq s_j, \forall s_i, s_j \in \tau = (s_0, \dots, s_n), i \neq j.$$

假设 2。 假设流量函数

$F(s, a)$ 是 Lipschitz 连续的，即

$$|F(s, a) - F(s, a')| \leq L \|a - a'\|, a, a' \in \mathcal{A}, \quad (3)$$

$$|F(s, a) - F(s', a)| \leq L \|s - s'\|, s, s' \in \mathcal{S}, \quad (4)$$

where L is a constant.

假设 3。 假设对于任意状态对 (s_t, s_{t+1}) ，存在唯一的动作 a_t 使得 $T(s_t, a_t) = s_{t+1}$ ，即在 s_t 采取动作 a_t 是到达 s_{t+1} 的唯一方式。因此我们可以定义 $s_t := g(s_{t+1}, a_t)$ ，其中 $g(\cdot)$ 是一个转移函数。并且假设动作是转换动作。

附录中分析了假设 1-3 的必要性和合理性。在假设 1 下，我们定义父集 $\mathcal{P}(s_t)$ 状态的 s_t 作为包含所有直接父节点的集合 s_t 这可以实现直接过渡到

s_t , i.e., $\mathcal{P}(s_t) = \{s \in \mathcal{S} : T(s, a \in \mathcal{A}) = s_t\}$ 类似地，定义子集 $\mathcal{C}(s_t)$ 状态的 s_t 作为该集合包含所有直接子节点的 s_t 这可以实现直接过渡到 s_t , i.e., $\mathcal{C}(s_t) = \{s \in \mathcal{S} : T(s_t, a \in \mathcal{A}) = s\}$ 然后，我们有以下连续流定义，其中假设 2-3 使得这些积分可积且有意义。定义 1 (连续状态流)。连续状态流 $F(s) : \mathcal{S} \mapsto \mathbb{R}$ 是通过该状态的完整轨迹流的积分：

$$F(s) = \int_{\tau: s \in \tau} F(\tau) d\tau. \quad (5)$$

定义 2 (连续流入)。 对于任意状态 s_t ，其流入是能够到达状态 s_t , i.e., 的流的积分

$$\int_{s \in \mathcal{P}(s_t)} F(s \rightarrow s_t) ds = \int_{s: T(s, a) = s_t} F(s, a) ds = F(s_t) = \int_{a: T(s_t, a) = s_t} F(s, a) da, \quad (6)$$

其中 $a : s \rightarrow s_t$ 和 $s = g(s_t, a)$ 因为假设 3 成立。定义 3 (连续流出)。对于任意状态 s_t ，其流出是通过状态 s_t 的所有可能动作 $a \in \mathcal{A}$, i.e., 的流的积分

$$\int_{s \in \mathcal{C}(s_t)} F(s_t \rightarrow s) ds = F(s_t) = \int_{a \in \mathcal{A}} F(s_t, a) da. \quad (7)$$

基于上述定义，我们可以将边 $s \rightarrow s'$ 的转移概率 $P(s \rightarrow s' | s)$ 定义为 Bengio 等人 (2021b) 引入的条件概率的一个特例。特别是，前向转移概率由

$$P_F(s_{t+1} | s_t) := P(s_t \rightarrow s_{t+1} | s_t) = \frac{F(s_t \rightarrow s_{t+1})}{F(s_t)}. \quad (8)$$

类似地，后向转移概率由

$$P_B(s_t|s_{t+1}) := P(s_t \rightarrow s_{t+1}|s_{t+1}) = \frac{F(s_t \rightarrow s_{t+1})}{F(s_{t+1})}. \quad (9)$$

对于从连续任务 (S, A) 中采样的任何轨迹，我们有

$$\forall \tau = (s_1, \dots, s_n), P_F(\tau) := \prod_{t=1}^{n-1} P_F(s_{t+1}|s_t) \quad (10)$$

$$\forall \tau = (s_1, \dots, s_n), P_B(\tau) := \prod_{t=1}^{n-1} P_B(s_t|s_{t+1}), \quad (11)$$

进一步地，我们有

$$\forall s \in S \setminus \{s_f\}, \int_{s' \in C(s)} P_F(s'|s) ds' = 1 \text{ 及 } \forall s \in S \setminus \{s_0\}, \int_{s' \in P(s)} P_B(s'|s) ds' = 1. \quad (12)$$

给定任何从 s_0 开始并在 s 结束的轨迹 $\tau = (s_0, \dots, s_n, s)$ ，马尔可夫流 (Bengio 等人, 2201b) 被定义为满足以下条件的流：

$$P(s \rightarrow s'|\tau) = P(s \rightarrow s'|s) = P_F(s'|s),$$

相应的流网络 (S, A, F) 被称为马尔可夫流网络 (Bengio 等人, 2021b)。然后，我们在附录 B.1 中证明了定理 1，它是 Bengio 等人 (2021b) 中命题 19 在连续场景下的扩展。

定理 1 (连续流匹配条件)。 考虑一个非负函数 $\hat{F}(s, a)$ ，它接受状态 $s \in S$ 和动作 $a \in A$ 作为输入。那么，我们有 $\hat{F}$ 对应于一个流当且仅当满足以下连续流匹配条件：

$$\begin{aligned} \forall s' > s_0, \hat{F}(s') &= \int_{s \in P(s')} \hat{F}(s \rightarrow s') ds = \int_{s: T(s, a) = s'} \hat{F}(s, a : s \rightarrow s') ds \\ \forall s' < s_f, \hat{F}(s') &= \int_{s'' \in C(s')} \hat{F}(s' \rightarrow s'') ds'' = \int_{a \in A} \hat{F}(s', a) da. \end{aligned} \quad (13)$$

此外， $\hat{F}$ 唯一地定义了一个与 $\hat{F}$ 匹配的马尔可夫流 F ，使得

$$F(\tau) = \frac{\prod_{t=1}^{n+1} \hat{F}(s_{t-1} \rightarrow s_t)}{\prod_{t=1}^n \hat{F}(s_t)}. \quad (14)$$

定理 1 意味着只要任何非负函数满足流匹配条件，就会确定一个唯一的流。因此，对于稀疏奖励环境，即 $R(s) = 0, \forall s \neq s_f$ ，我们可以通过训练一个满足流匹配条件的流网络来获得目标流。这样的学习机被称为 CFlowNets，我们有以下连续损失函数：

$$\mathcal{L}(\tau) = \sum_{s_t=s_1}^{s_f} \left(\int_{s_{t-1} \in P(s_t)} F(s_{t-1} \rightarrow s_t) ds_{t-1} - R(s_t) - \int_{s_{t+1} \in C(s_t)} F(s_t \rightarrow s_{t+1}) ds_{t+1} \right)^2.$$

然而，上述连续损失函数显然不能直接应用于实际操作。接下来，我们提出一种基于采样轨迹近似连续损失函数的方法，从而获得流模型。

4 CFLOWNETS: 训练框架

对于连续任务，通常难以访问所有状态-动作对来计算连续流入和流出。在下文中，我们提出了 CFlowNets 训练框架来解决这个问题，该框架包括一个动作采样过程和一个流匹配近似过程。然后，可以基于近似的流匹配损失函数来训练 CFlowNets。

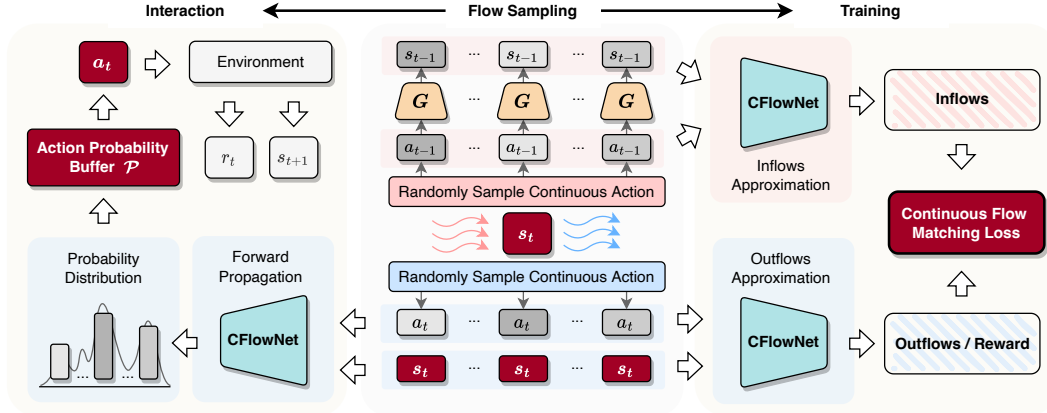


图 1: CFlowNets 的整体框架。左: 在环境交互阶段, 我们根据 CFlowNet 的概率按奖励比例采样动作以更新状态。中: 我们随机采样动作以近似计算流入和流出, 其中使用深度神经网络估计父状态。右: 基于使流入等于流出或奖励的连续流匹配损失来训练 CFlowNet。

4.1 整体框架

CFlowNets 的概述框架如图 1 所示, 包括环境交互、流采样和训练过程。在环境交互阶段 (图 1 左侧部分), 我们基于 CFlowNets 的前向传播采样动作概率缓冲区。我们将此过程称为动作选择过程, 具体细节见第 4.2 节。获取动作后, 代理可以与环境互动以更新状态, 这一过程重复若干步直到完整轨迹被采样。一旦完整的轨迹缓冲区可用, 我们就随机采样 K 动作并计算子状态以近似计算流出。对于流入, 我们使用这些采样的动作连同当前状态作为深度神经网络 G 的输入来估计父状态。基于这些, 我们可以近似确定流入。我们将此过程称为流匹配近似过程 (图 1 中间部分), 具体细节见第 4.3 节。最后, 基于近似的流入和流出, 我们可以基于连续流匹配损失函数训练 CFlowNet (图 1 右侧部分), 具体细节见第 4.4 节。伪代码见附录 C。

4.2 动作选择过程

从空集开始, CFlowNets 旨在通过迭代采样获得完整的轨迹 $\tau = (s_0, s_1, \dots, s_f) \in \mathcal{T}$, 每次采样都与元组相关联 $a_t \sim \pi(a_t|s_t) = \frac{F(s_t, a_t)}{F(s_t)}$ 。然而, 由于动作是连续的, 严格根据相应的概率进行轨迹采样较为困难 $\{(s_t, a_t, r_t, s_{t+1})\}_{t=0}^f$, 我们无法基于流网络得到精确的动作概率分布函数 a_t , 因此存在挑战 $F(s_t, a_t)$ 。为解决此问题, 在每个状态 s_t , 我们首先均匀采样 M 动作来自 $\mathcal{A}$ 并生成一个动作概率缓冲区 $\mathcal{P} = \{F(s_t, a_i)\}_{i=1}^M$, 将其作为动作概率分布的近似值。然后根据所有动作对应的概率从 $\mathcal{P}$ 中采样一个动作。显然, 具有较大 $F(s_t, a_i)$ 的动作将以更高的概率被采样。这样, 我们就能够根据其相应概率从连续分布中近似采样动作。

注记 1. 训练过程结束后, 对于需要较大奖励的任务, 在测试过程中可以通过采样具有最大流模型输出的动作 $\mathcal{P}$ 来获得相对较高的奖励。流模型输出的使用方式灵活, 我们可以根据不同的任务进行调整。

4.3 流匹配近似

一旦一批轨迹 $\mathcal{B}$ 可用, 为了满足流量条件, 要求对于任意节点 s_t , 流入

$$\int_{a: T(s, a) = s_t} F(s, a) da \quad \int_{a \in \mathcal{A}} F(s_t, a) da, \text{ 即总流量}$$

量等于流出量

节点 s_t 的 $F(s_t)$ 。然而，显然，我们不能直接计算连续流入和流出以完成流量匹配条件。一个直观的想法是基于合理的近似离散化流入和流出，并匹配离散化的流量。为此，我们从连续动作空间 $\mathcal{A}$ 中独立均匀地采样 K 个动作，并计算相应的 $F(s_t, a_k), k = 1, \dots, K$ 作为流出，即我们使用以下近似：

$$\int_{a \in \mathcal{A}} F(s_t, a) da \approx \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(s_t, a_k), \quad (15)$$

其中 $\mu(\mathcal{A})$ 表示连续动作空间的测度 $\mathcal{A}$ 。

相比之下，近似流入更为困难，因为我们首先需要找到父状态。为了解决这个问题，我们构建了一个深度神经网络 G （命名为“检索”神经网络），该网络由 ϕ 参数化，输入为 (s_{t+1}, a_t) ，输出为 s_t ，并基于 $\mathcal{B}$ 使用均方误差损失函数来训练这个网络。也就是说，我们希望使用 G 来拟合函数 $g(\cdot)$ 。通常，网络 G 容易训练，因为我们考虑的任务满足假设 3，并且我们可以通过简单的预训练获得高精度的网络 G 。随着训练的进行，我们还可以偶尔根据采样的轨迹更新 G 以确保准确性。然后，可以近似计算流入量：

$$\int_{a: T(g(s_t, a), a) = s_t} F(g(s_t, a), a) da \approx \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(G_\phi(s_t, a_k), a_k). \quad (16)$$

接下来，通过假设流函数 $F(s, a)$ 在假设 2 中是 Lipschitz 连续的，我们可以提供样本流入/流出与真实流入/流出之间误差的非渐进分析。定理 2 建立了样本流出（相应地流入）与实际流出（相应地流入）之间的误差界限，并表明尾部呈指数级下降。此外，随着 K 的增加，尾部变得非常小，这意味着样本流出（相应地流入）是对实际流出（相应地流入）的良好估计。

定理 2. 设 $\{a_k\}_{k=1}^K$ 独立均匀地从连续动作空间 $\mathcal{A}$ 中抽取。假设 G_{ϕ^*} 能够最优地输出实际状态 s_t ，条件是 (s_{t+1}, a_t) 。对于任何有界的连续动作 $a \in \mathcal{A}$ 和任何状态 $s_t \in \mathcal{S}$ ，我们有

$$\mathbb{P} \left(\left| \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(s_t, a_k) - \int_{a \in \mathcal{A}} F(s_t, a) da \right| \geq t \right) \leq 2 \exp \left(-\frac{Kt^2}{2(L\mu(\mathcal{A})\text{diam}(\mathcal{A}))^2} \right) \quad (17)$$

以及

$$\mathbb{P} \left(\left| \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(G_{\phi^*}(s_t, a_k), a_k) - \int_{a: T(s, a) = s_t} F(s, a) da \right| \geq t \right) \leq 2 \exp \left(-\frac{Kt^2}{2(L\mu(\mathcal{A})(\text{直径}(\mathcal{A}) + \text{直径}(\mathcal{S})))^2} \right), \quad (18)$$

其中 L 是 Lipschitz 常数， $\text{直径}(\mathcal{A})$ 表示动作空间 $\mathcal{A}$ 的直径，而 $\text{直径}(\mathcal{S})$ 表示状态空间 $\mathcal{S}$ 的直径。

4.4 损失函数

基于 (15) 和 (16)，连续损失函数可以近似为

$$\mathcal{L}_\theta(\tau) = \sum_{s_t = s_1}^{s_f} \left[\sum_{k=1}^K F_\theta(G_\phi(s_t, a_k), a_k) - \lambda R(s_t) - \sum_{k=1}^K F_\theta(s_t, a_k) \right]^2, \quad (19)$$

其中 θ 是流网络 $F(\cdot)$ 的参数，并且是 $\lambda = K/\mu(\mathcal{A})$ 。请注意，在许多任务中我们无法获得确切的 $\mu(\mathcal{A})$ 。对于此类任务，我们可以直接将 λ 设置为 1，然后调整奖励塑形以确保算法 1 的收敛性。

1A 常用的奖励塑形方法是将奖励乘以一个常数，并调整奖励到适当的范围内，以确保更好的收敛。因此，在将 λ 设置为 1 后，合理的奖励塑形操作也可以补偿 λ 误差的影响。

值得注意的是，轨迹中不同位置的状态流的大小可能不匹配。例如，初始节点的流可能大于结束节点的流。为了解决这个问题，受到 GFlowNets (Bengio 等, 2021a) 引入的对数尺度损失的启发，我们可以将 (19) 修改为：

$$\mathcal{L}_\theta(\tau) = \sum_{s_t=s_1}^{s_f} \left\{ \log \left[\epsilon + \sum_{k=1}^K \exp F_\theta^{\log}(G_\phi(s_t, a_k), a_k) \right] - \log \left[\epsilon + \lambda R(s_t) + \sum_{k=1}^K \exp F_\theta^{\log}(s_t, a_k) \right] \right\}^2, \quad (20)$$

其中， ϵ 是一个超参数，有助于权衡小流和大流之间的关系，并帮助避免取对数时数值过小的问题。需要注意的是，定理 2 不能用于保证 (20) 的无偏性，因为 $\log \mathbb{E}(x) \neq \mathbb{E} \log(x)$ 。但实验表明，这种近似效果良好。

5 相关工作

生成流网络。 生成流网络被提出以增强探索能力，通过生成一个与终止状态上的奖励成比例的分布 (Bengio 等, 2021b;a)。由于该网络仅根据相应奖励的分布采样动作，而不是像强化学习那样专注于最大化奖励的动作，因此它在具有更多样化奖励分布的任务上表现良好，并已成功应用于分子生成 (Bengio 等, 2021a; Malkin 等, 2022; Jain 等, 2022)、离散概率建模 (Zhang 等, 2022b)、结构学习 (Deleu 等, 2022)、因果发现 Li 等 (2022) 和图神经网络 Li 等 (2023)。Zhang 等 (2022a) 通过马尔可夫轨迹学习的角度讨论了深度生成模型与 GFlowNets 之间的联系。在 Bengio 等 (2021b) 中，提出了将 GFlowNets 适应于连续任务的想法，即通过用积分替换连续变量的求和。Malkin 等 (2022) 和 Bengio 等 (2021b) 提出了详细平衡 (DB) 和轨迹平衡 (TB) 目标，这些目标在目标函数中使用参数化的正向和反向策略。这些新目标函数不需要评估状态多个父节点上的流模型，这在高维环境中更为高效。Malkin 等 (2022) 和 Bengio 等 (2021b) 提到，这些目标函数也可以通过将目标中的策略似然替换为概率密度来用于连续场景。一个可能的劣势是，在连续环境中估计 P_F 和 P_B 并不容易，因为状态空间比离散场景大得多，而对概率密度建模的小误差可能会极大地影响最终性能。如何结合 DB 和 TB 与 CFlowNets 将是未来值得研究的工作。

连续强化学习。 策略梯度算法广泛用于具有连续动作空间的强化学习问题。确定性策略梯度 (DPG) (Silver 等, 2014) 算法是一种演员-评论家方法 (Grondman 等, 2012; Rosenstein 等, 2004)，它使用学习值的估计 $Q(s, a)$ 来训练由 θ^μ 参数化的确定性策略 $\mu: \mathcal{S} \rightarrow \mathcal{A}$ 。与 CFlowNets 相比，策略通过将链式法则应用于从起始分布开始的预期回报 J 相对于策略参数进行更新：

$$\begin{aligned} \nabla_{\theta^\mu} J &\approx \mathbb{E}_{\mathcal{D}} \left[\nabla_{\theta^\mu} Q(s, a | \theta^\mu) \Big|_{a=\mu(s|\theta^\mu)} \right] \\ &= \mathbb{E}_{\mathcal{D}} \left[\nabla_a Q(s, a | \theta^\mu) \Big|_{a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) \right], \end{aligned} \quad (21)$$

其中 $\mathcal{D}$ 是回放缓冲区。策略旨在最大化未来奖励的期望，这些奖励通过 Q 学习进行估计。在这种设置下，由策略生成的轨迹可能相对同质。然而，CFlowNets 的训练目标是定义一个与终止状态上的奖励成比例的分布，从而产生更多样化的轨迹，有利于探索环境。

后来，深度 DPG (DDPG) (Lillicrap 等, 2015 年) 改进了 DPG，并具有良好的样本效率，但存在极端脆弱性和超参数敏感性。因此，难以将 DDPG 扩展

到复杂的高维任务。为了改进 DDPG，双延迟 DDPG (TD3) (Fujimoto 等, 2018 年) 采用了演员-评论家框架，并考虑了价值更新和函数逼近误差在策略中的相互作用。还有一些基于策略梯度 (Sutton 等, 1999 年; Kohl & Stone, 2004 年; Khadka & Tumer, 2018 年) 的算法可以适应连续任务，如近端策略优化 (PPO) (Schulman 等, 2017 年) 算法、异步优势演员-评论家 (A3C) (Stooke & Abbeel, 2018 年) 以及重要加权演员-学习者架构 (IMPALA) (Espeholt 等, 2018 年)。PPO 具有信赖域策略优化 (Schulman 等, 2015 年) 的优势，允许多批数据一起更新。因此，它更易于实现，更具通用性，并且具有较低的样本复杂度。最近，阶段性策略梯度 (PPG) (Cobbe 等, 2021 年) 被提出，用于解耦策略和价值函数之间的训练，同时保持特征共享，PPG 通过适当水平的样本重用优化每个目标，以提高样本效率。大多数这些改进的策略梯度方法都可以归类为旨在最大化奖励，因此它们都不比 CFlowNets 更适合探索任务。

此外，一些基于最大熵 (Pitis 等, 2020; Haarnoja 等, 2018a; Hazan 等, 2019; Yarats 等, 2021) 的强化学习算法也可以适应连续任务，例如软演员-评论家 (SAC) (Haarnoja 等, 2018b)。通过最大化预期奖励和熵，SAC 的演员网络可以在尽可能随机地行动的同时成功完成任务。CFlowNets 与 SAC 的区别在于：1) SAC 通过高斯策略选择动作，这比使用一般的未归一化的动作概率密度函数 $F(s, a)$ 的表达能力弱；2) 在一般情况下，SAC 学习与长期回报成比例，生成满足条件的轨迹分布 $p(\tau) \propto R(\tau)$ 其中 $R(\tau)$ 是回报值 τ 。CFlowNets 考虑所有可能通向终止状态的轨迹 s_f ，并学习生成策略以满足条件 s_f 其中 $p(s_f) \propto R(s_f)$ 。

6 实验

为了展示所提出的 CFlowNets 的有效性，我们在几个具有稀疏奖励的连续控制任务上进行了实验，包括 Point-Robot-Sparse、Reacher-Goal-Sparse 和 Swimmer-Sparse。这些环境的可视化结果如图 7、图 8 和图 9 所示。然后将 CFlowNets 与几种最先进基线 RL 算法进行比较，例如 DDPG (Lillicrap 等, 2015)、TD3 (Fujimoto 等, 2018)、PPO (Schulman 等, 2017) 和 SAC (Haarnoja 等, 2018b)。更多实现细节见附录 D。

图 2 展示了 CFlowNets 和 RL 算法学习策略的分布情况。所有曲线都经过了最大最小归一化处理。灰色曲线表示当智能体前往坐标 (7, 7) 时，不同动作产生的奖励分布的真实情况，表明此处的最佳动作是向右或向上移动。红色曲线显示了在不同动作下 CFlowNets 的流网络输出，表明 CFlowNets 对奖励有很好的拟合能力。相比之下，其他强化学习算法难以很好地拟合实际的奖励分布。

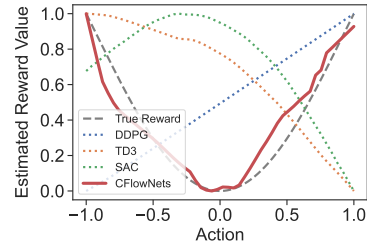


图 2: Point-Robot-Sparse 任务上的奖励分布。

图 3(a)-(c) 分别展示了在 Point-Robot-Sparse、Reacher-Goal-Sparse 和 Swimmer-Sparse 环境中，随着训练的进行所探索的有效独特轨迹的数量。经过一定数量的训练轮次后，收集了 10000 条轨迹。有效独特轨迹被定义为奖励高于一个阈值 δ_r ，同时该轨迹与其他轨迹之间的均方误差大于另一个阈值 δ_{mse} 。也就是说，如果两条轨迹的回报都很高，但两者接近且均方误差较小，则认为这是一条有效独特探索。在 Point-Robot-Sparse、Reacher-Goal-Sparse 和 Swimmer-Sparse 中， δ_r 分别设置为 0.5、-0.2、5.0。在 Point-Robot-Sparse、Reacher-Goal-Sparse 和 Swimmer-Sparse 中， δ_{mse} 分别设置为 0.02、4.0、1.0。从图中可以看出，DDPG、TD3 和 PPO 的探索能力最差，仅生成了一条有效独特轨迹。SAC 在训练初期探索较好，但随着训练的进行逐渐减少并趋于收敛。相比之下，CFlowNets 的探索能力非常突出，所探索的轨迹数量远超其他算法，并且随着训练的进行，其探索能力保持稳定。

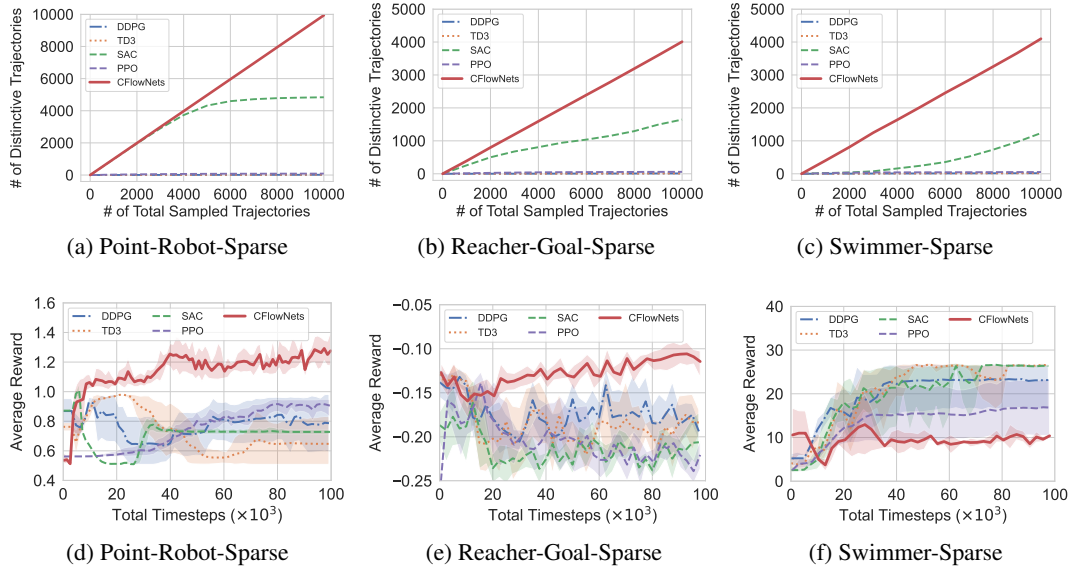


图 3: CFlowNets、DDPG、TD3、SAC 和 PPO 在 Point-Robot-Sparse、Reacher-Goal-Sparse 和 Swimmer-Sparse 任务上的比较结果。上部：在 10000 次探索下生成的有效独特轨迹数量。下部：不同方法的平均奖励。

图 3(d)-(f) 分别显示了在 Point-Robot-Sparse、Reacher-Goal-Sparse 和 Swimmer-Sparse 环境中的训练过程中所获得的奖励。阴影区域表示 5 次运行的 95% 置信区间。图 3(d) 和图 3(e) 表明，CFlowNets 具有最快且更稳定的上升趋势，最终奖励远超其他算法。相比之下，在图 3(f) 中，CFlowNets 的表现不如其他算法。由于 Point-Robot-Sparse 和 Reacher-Goal-Sparse 的奖励分布较为均匀，因此这两个任务更倾向于探索。CFlowNets 具有更好的探索能力，从而能够稳定收敛。至于 Swimmer-Sparse，其奖励分布相对陡峭，接近最大奖励的采样可以实现更快的收敛。因此，CFlowNets 在该任务上的奖励表现不如强化学习是合理的。然而，在此环境中，CFN 仍能保持良好的探索能力。

7 结论

本文提出了生成连续流网络以增强连续控制任务中的探索能力。首先介绍了 CFlowNets 的理论公式，然后提出了 CFlowNets 的训练框架，包括动作选择过程、流近似算法以及连续流匹配损失函数。理论分析表明，随着流样本数量的增加，流近似的误差迅速减小。实验结果表明，在连续控制任务中，CFlowNets 相比许多强化学习方法具有性能优势，尤其是在探索能力方面，CFlowNets 的效果远远超过其他最先进强化学习算法。

局限性: 与 GFlowNets 类似，CFlowNets 旨在根据流网络进行动作采样，而不是选择最大化奖励的动作。因此，CFlowNets 更适合于偏向探索的任务。在旨在最大化奖励的任务上，CFlowNets 的表现不如强化学习。当然，CFlowNets 的目的不是完全取代强化学习，而是作为强化学习的补充，为连续控制任务提供一个新的选项。未来工作：未来的工作将是如何结合 CFlowNets 与 DB 和 TB 目标函数以提高训练效率。

参考文献

- Dan Alistarh, Torsten Hoefer, Mikael Johansson, Nikola Konstantinov, Sarit Khirirat, and Cédric Renggli. The convergence of sparsified gradient methods. *Advances in Neural Information Processing Systems*, 31, 2018.
- Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. In *International Conference on Machine Learning*, pp. 242–252. PMLR, 2019.
- OpenAI: Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.
- Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow network based generative models for non-iterative diverse candidate generation, 2021a.
- Yoshua Bengio, Tristan Deleu, Edward J. Hu, Salem Lahlou, Mo Tiwari, and Emmanuel Bengio. Gflownet foundations, 2021b.
- Karl W Cobbe, Jacob Hilton, Oleg Klimov, and John Schulman. Phasic policy gradient. In *International Conference on Machine Learning*, pp. 2020–2027. PMLR, 2021.
- Tristan Deleu, António Góis, Chris Emezue, Mansi Rankawat, Simon Lacoste-Julien, Stefan Bauer, and Yoshua Bengio. Bayesian structure learning with generative flow networks. *arXiv preprint arXiv:2202.13903*, 2022.
- Simon Du, Jason Lee, Haochuan Li, Liwei Wang, and Xiyu Zhai. Gradient descent finds global minima of deep neural networks. In *International conference on machine learning*, pp. 1675–1685. PMLR, 2019.
- Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International conference on machine learning*, pp. 1407–1416. PMLR, 2018.
- Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pp. 1587–1596. PMLR, 2018.
- Ivo Grondman, Lucian Busoniu, Gabriel AD Lopes, and Robert Babuska. A survey of actor-critic reinforcement learning: Standard and natural policy gradients. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 42(6):1291–1307, 2012.
- Tuomas Haarnoja, Kristian Hartikainen, Pieter Abbeel, and Sergey Levine. Latent space policies for hierarchical reinforcement learning. In *International Conference on Machine Learning*, pp. 1851–1860. PMLR, 2018a.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. PMLR, 2018b.
- Elad Hazan, Sham Kakade, Karan Singh, and Abby Van Soest. Provably efficient maximum entropy exploration. In *International Conference on Machine Learning*, pp. 2681–2691. PMLR, 2019.
- Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.
- Moksh Jain, Emmanuel Bengio, Alex Hernandez-Garcia, Jarrid Rector-Brooks, Bonaventure FP Dossou, Chanakya Ajit Ekbote, Jie Fu, Tianyu Zhang, Michael Kilgour, Dinghuai Zhang, et al. Biological sequence design with gflownets. In *International Conference on Machine Learning*, pp. 9786–9801. PMLR, 2022.
- Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 4:237–285, 1996.

- Shauharda Khadka and Kagan Tumer. Evolution-guided policy gradient in reinforcement learning. *Advances in Neural Information Processing Systems*, 31, 2018.
- B Ravi Kiran, Ibrahim Sobh, Victor Talpaert, Patrick Mannion, Ahmad A Al Sallab, Senthil Yogamani, and Patrick Pérez. Deep reinforcement learning for autonomous driving: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- Nate Kohl and Peter Stone. Policy gradient reinforcement learning for fast quadrupedal locomotion. In *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004*, volume 3, pp. 2619–2624. IEEE, 2004.
- Wenqian Li, Yinchuan Li, Shengyu Zhu, Yunfeng Shao, Jianye Hao, and Yan Pang. Gflowcausal: Generative flow networks for causal discovery. *arXiv preprint arXiv:2210.08185*, 2022.
- Wenqian Li, Yinchuan Li, Zhigang Li, Jianye HAO, and Yan Pang. Dag matters! gflownets enhanced explainer for graph neural networks. In *International conference on learning representations*, 2023.
- Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in gflownets. *arXiv preprint arXiv:2201.13259*, 2022.
- Xinlei Pan, Yurong You, Ziyang Wang, and Cewu Lu. Virtual to real reinforcement learning for autonomous driving. *arXiv preprint arXiv:1704.03952*, 2017.
- Silviu Pitis, Harris Chan, Stephen Zhao, Bradly Stadie, and Jimmy Ba. Maximum entropy gain exploration for long horizon multi-goal reinforcement learning. In *International Conference on Machine Learning*, pp. 7750–7761. PMLR, 2020.
- Michael T Rosenstein, Andrew G Barto, Jennie Si, Andy Barto, Warren Powell, and Donald Wunsch. Supervised actor-critic reinforcement learning. *Learning and Approximate Dynamic Programming: Scaling Up to the Real World*, pp. 359–380, 2004.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pp. 1889–1897. PMLR, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shai Shalev-Shwartz, Shaked Shammah, and Amnon Shashua. Safe, multi-agent, reinforcement learning for autonomous driving. *arXiv preprint arXiv:1610.03295*, 2016.
- David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pp. 387–395. PMLR, 2014.
- Adam Stooke and Pieter Abbeel. Accelerated methods for deep reinforcement learning. *arXiv preprint arXiv:1803.02811*, 2018.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.
- Roman Vershynin. *High-dimensional probability: An introduction with applications in data science*, volume 47. Cambridge university press, 2018.
- Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Reinforcement learning with prototypical representations. In *International Conference on Machine Learning*, pp. 11920–11931. PMLR, 2021.

Dinghuai Zhang, Ricky TQ Chen, Nikolay Malkin, and Yoshua Bengio. Unifying generative models with gflownets. *arXiv preprint arXiv:2209.02606*, 2022a.

Dinghuai Zhang, Nikolay Malkin, Zhen Liu, Alexandra Volokhova, Aaron Courville, and Yoshua Bengio. Generative flow networks for discrete probabilistic modeling. *arXiv preprint arXiv:2202.01361*, 2022b.

附录 讨论

A.1 为什么假设 1 是必要且合理的?

必要性: 在大多数环境中, 当在一个连续空间中采样轨迹时, 生成循环是很困难的。由于 $\forall t, \mu(\{s_0, \dots, s_t\}) = 0$ 和 $\mu(\mathcal{A}) = \mu(\mathcal{A} \setminus \{s_0, \dots, s_t\})$, 即概率 $s_{t+1} \in \{s_0, \dots, s_t\}$ 非常小。然而, 当某些环境具有特殊约束时, 循环经常会出现。例如, 一个简单的摆动任务 (见图 4), 动作是控制摆从当前位置旋转到下一个位置, 在某个角度上。对于这个任务, 在连续空间中很难使摆旋转到完全相同的位置。然而, 如果在任务中添加一面墙, 摆很容易到达相同的位置 (见图 5), 即会发生循环。因此, 我们仍然需要添加一个无环假设, 以确保 CFlowNets 的理论和性能得到保证。

合理性: 这一假设是合理的, 因为对于许多连续环境来说, 在没有特殊约束的情况下, 形成轨迹循环是很困难的。即使是容易形成循环的任务, 我们也可以直接在状态空间中添加时间步来满足这一假设。



图 4: 摆动。在这个连续空间中, 状态很难完全一致。

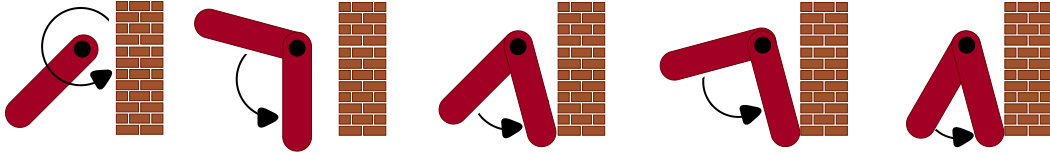


图 5: 带墙的摆动。到达墙壁时, 状态变得一致。

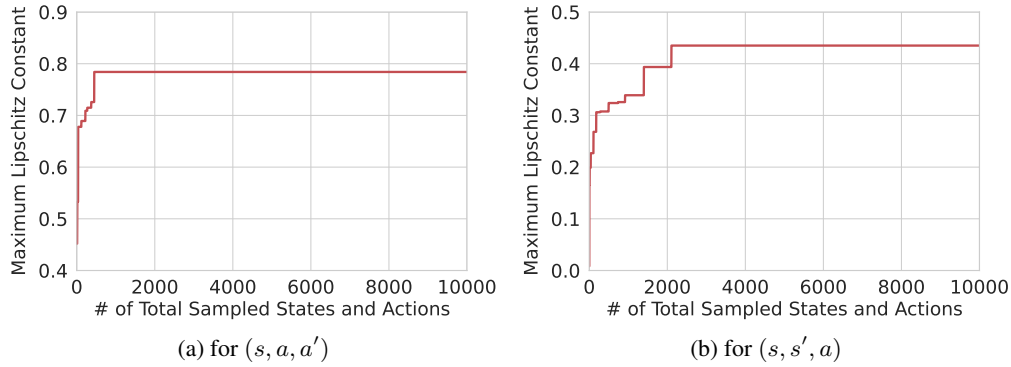


图 6: 流网络 $F(s, a)$ 的累积最大 Lipschitz 常数。

A.2 为什么假设 2 是必要的并且合理的?

必要性: 这一假设主要用于保证与流相关的积分的存在性, 并确保定理 2 成立。

合理性: 我们基于仿真来验证这一假设。如图 6 所示, 我们计算 $\frac{|F(s, a) - F(s', a)|}{\|s - s'\|}$ 和 $\frac{|F(s, a) - F(s, a')|}{\|a - a'\|}$ 每个样本元组 (s, a, a') 和 (s, s', a) 的 Lipschitz 常数, 分别进行分析。它们累积的最大 Lipschitz 常数分别显示在图 6(a) 和 (b) 中。显然, 我们的流网络存在有限的 Lipschitz 常数。

此外, Lipschitz 连续是神经网络的一个常见假设, 这里举一些快速的例子: 杜等人 (2019); 雅科特等人 (2018); 艾伦-朱等人 (2019); 阿利斯塔尔等人 (2018) 都使用这一假设来证明算法的收敛性。

A.3 为什么假设 3 是必要的并且合理的?

必要性: 这一假设用于定义 2, 并使检索神经网络能够拟合函数 $g(s, a)$ 。尽管大多数环境状态转换与动作之间存在一一对应关系, 但仍有一些特殊情况, 在给定一个状态对 (s, s') 的情况下, 可能存在无限数量的动作。例如: 对于图 5 中的带墙摆锤, 到达墙壁后, 继续增加动作不会继续改变状态 s' 。此外, 平移动作的特殊情况可能是 $T(s, a) = s + a$ 或者使用特殊线性群, 使得定义 2 和 3 成立。平移动作用于确保在连续流定义中没有雅可比项。

合理性: 这一假设是许多环境的特性, 因此是合理的。对于不满足此假设的环境, 我们可以通过修改状态来添加更多信息以使其满足该假设。例如, 我们可以将动作持续时间添加到图 5 所示的带墙摆动任务的状态空间中。即使在到达墙壁后动作增加, 位置信息也不会改变, 但持续时间会增加, 从而使状态转换与动作一一对应。最坏的情况是我们无法更改环境以满足该假设。此时, 我们需要解决的问题是, 当输入固定时, 检索神经网络 G 的输出不能为多个值。我们的一个猜想是, 也许通过向输入添加一些小的随机噪声可以缓解这个问题, 但这一想法尚未经过验证。

B 证明

B.1 定理 1 的证明

定理 1. (连续流匹配条件)。考虑一个非负函数 $\hat{F}(s, a)$, 其输入是一个状态 $s \in \mathcal{S}$ 和一个动作 $a \in \mathcal{A}$ 。那么, $\hat{F}$ 对应于一个流当且仅当满足以下连续流匹配条件:

$$\begin{aligned} \forall s' > s_0, \hat{F}(s') &= \int_{s \in \mathcal{P}(s')} \hat{F}(s \rightarrow s') ds = \int_{s: T(s, a) = s'} \hat{F}(s, a : s \rightarrow s') ds \\ \forall s' < s_f, \hat{F}(s') &= \int_{s'' \in \mathcal{C}(s')} \hat{F}(s' \rightarrow s'') ds'' = \int_{a \in \mathcal{A}} \hat{F}(s', a) da. \end{aligned} \quad (22)$$

此外, $\hat{F}$ 唯一地定义了一个与 $\hat{F}$ 匹配的马尔可夫流 F , 使得

$$F(\tau) = \frac{\prod_{t=1}^{n+1} \hat{F}(s_{t-1} \rightarrow s_t)}{\prod_{t=1}^n \hat{F}(s_t)}. \quad (23)$$

证明。该证明是对 Bengio 等人 (2021b) 提出的命题 19 在连续情况下的扩展。首先, 我们证明必要性部分。给定一个流网络, 在轨迹上的非初始和非最终节点处, 通过状态 s' 的所有完整轨迹集合是所有动作 $s \in \mathcal{P}(s')$ 下通过 $s \rightarrow s'$ 的轨迹集合的并集, 同时也是所有状态 $s'' \in \mathcal{C}(s')$, i.e., 下通过 $s' \rightarrow s''$ 的轨迹集合的并集。

$$\{\tau \in \mathcal{T} : s' \in \tau\} = \bigcup_{s \in \mathcal{P}(s')} \{\tau \in \mathcal{T} : s \rightarrow s' \in \tau\} = \bigcup_{s'' \in \mathcal{C}(s')} \{\tau \in \mathcal{T} : s' \rightarrow s'' \in \tau\}.$$

然后我们有

$$F(s') = \int_{\tau: s' \in \tau} F(\tau) d\tau = \int_{s \in \mathcal{P}(s')} \int_{\tau: s \rightarrow s' \in \tau} F(\tau) d\tau ds = \int_{s \in \mathcal{P}(s')} F(s \rightarrow s') ds,$$

以及

$$F(s') = \int_{\tau: s' \in \tau} F(\tau) d\tau = \int_{s'' \in \mathcal{C}(s')} \int_{\tau: s' \rightarrow s'' \in \tau} F(\tau) d\tau ds'' = \int_{s'' \in \mathcal{C}(s')} F(s' \rightarrow s'') ds''.$$

接着我们完成了必要性的证明。接下来我们展示充分性。设 $\hat{Z} = \hat{F}(s_0)$ 为划分函数， $\hat{P}_F$ 为前向概率函数，则存在唯一的马尔可夫流 F ，其前向转移概率函数为 $P_F = \hat{P}_F$ ，划分函数为 Z ，根据 Bengio 等人 (2021b) 提出的命题 18，并且满足

$$F(\tau) = \hat{Z} \prod_{t=1}^{n+1} \hat{P}_F(s_t | s_{t-1}) = \frac{\prod_{t=1}^{n+1} \hat{F}(s_{t-1} \rightarrow s_t)}{\prod_{t=1}^n \hat{F}(s_t)}, \quad (24)$$

其中 $s_{n+1} = s_f$ 。此外，根据引理 1，我们有

$$\int_{\tau \in \mathcal{T}_{0,s}} P_B(\tau) d\tau = \int_{\tau \in \mathcal{T}_{0,s}} \prod_{s_t \rightarrow s_{t+1} \in \tau} P_B(s_t | s_{t+1}) d\tau = 1.$$

引理 1. 考虑一个连续任务 $(\mathcal{S}, \mathcal{A})$ ，其中我们定义了转移概率 (8) 和 (9)。定义 $\mathcal{T}_{s,f}$ 和 $\mathcal{T}_{0,s}$ 分别为从状态 s 开始并在状态 s_f 结束，以及从状态 s_0 开始并在状态 s 结束的轨迹集合。那么我们有

$$\forall s \in \mathcal{S} \setminus \{s_f\}, \int_{\tau \in \mathcal{T}_{s,f}} P_F(\tau) d\tau = 1 \quad (25)$$

$$\forall s \in \mathcal{S} \setminus \{s_0\}, \int_{\tau \in \mathcal{T}_{0,s}} P_B(\tau) d\tau = 1. \quad (26)$$

Thus, we have for $s' \neq s_0$:

$$\begin{aligned} F(s') &= \hat{Z} \int_{\tau \in \mathcal{T}_{0,s'}} \prod_{(s_t \rightarrow s_{t+1}) \in \tau} \hat{P}_F(s_{t+1} | s_t) d\tau \\ &= \hat{Z} \frac{\hat{F}(s')}{\hat{F}(s_0)} \int_{\tau \in \mathcal{T}_{0,s'}} \prod_{(s_t \rightarrow s_{t+1}) \in \tau} \hat{P}_B(s_t | s_{t+1}) d\tau = \hat{F}(s'). \end{aligned} \quad (27)$$

将公式 (27) 与 $P_F = \hat{P}_F$ 结合得到 $\forall s \rightarrow s' \in \mathcal{A}$, $F(s \rightarrow s') = \hat{F}(s \rightarrow s')$ 。最后，根据 Bengio 等人的命题 16 (2021b)，对于任意马尔可夫流 F' 匹配状态和边的关系 $\hat{F}$ ，我们有 $F'(\tau) = F(\tau)$ ，这表明了唯一性性质。然后我们完成证明。□

B.2 定理 2 的证明

定理 2. 设 $\{a_k\}_{k=1}^K$ 独立均匀地从连续动作空间 $\mathcal{A}$ 中抽取。假设 G_{ϕ^*} 能够最优地输出实际状态 s_t ，条件是 (s_{t+1}, a_t) 。对于任何有界的连续动作 $a \in \mathcal{A}$ 和任何状态 $s_t \in \mathcal{S}$ ，我们有

$$\mathbb{P} \left(\left| \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(s_t, a_k) - \int_{a \in \mathcal{A}} F(s_t, a) da \right| \geq t \right) \leq 2 \exp \left(-\frac{Kt^2}{2(L\mu(\mathcal{A})\text{diam}(\mathcal{A}))^2} \right) \quad (28)$$

以及

$$\begin{aligned} \mathbb{P} \left(\left| \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(G_{\phi^*}(s_t, a_k), a_k) - \int_{a: T(s, a) = s_t} F(s, a) da \right| \geq t \right) \\ \leq 2 \exp \left(-\frac{Kt^2}{2(L\mu(\mathcal{A})(\text{直径}(\mathcal{A}) + \text{直径}(\mathcal{S})))^2} \right), \end{aligned} \quad (29)$$

其中 L 是 Lipschitz 常数， $\text{直径}(\mathcal{A})$ 表示动作空间 $\mathcal{A}$ 的直径，而 $\text{直径}(\mathcal{S})$ 表示状态空间 $\mathcal{S}$ 的直径。

证明。首先，我们证明样本流出的期望是真实的流出，样本流入的期望是真实的流入，这在引理 2 中给出。

引理 2. 设 $\{a_k\}_{k=1}^K$ 是从连续动作空间 $\mathcal{A}$ 中独立均匀采样的。假设 G_{ϕ^*} 能够最优地输出实际状态 s_t ，并且使用 (s_{t+1}, a_t) 。那么对于任意状态 $s_t \in \mathcal{S}$ ，我们有

$$\mathbb{E} \left[\frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(s_t, a_k) \right] = \int_{a \in \mathcal{A}} F(s_t, a) da \quad (30)$$

以及

$$\mathbb{E} \left[\frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(G_{\phi^*}(s_t, a_k), a_k) \right] = \int_{a: T(s, a) = s_t} F(s, a) da, \quad (31)$$

其中 $s = g(s_t, a)$ 。

然后，定义以下术语：

$$\Gamma_k = \frac{\mu(\mathcal{A})}{K} F(s_t, a_k) - \frac{1}{K} \int_{a \in \mathcal{A}} F(s_t, a) da = \frac{1}{K} \int_{a \in \mathcal{A}} [F(s_t, a_k) - F(s_t, a)] da \quad (32)$$

以及

$$\Lambda_k = \frac{\mu(\mathcal{A})}{K} F(G_{\phi^*}(s_t, a_k), a_k) - \frac{1}{K} \int_{a: T(s, a) = s_t} F(s, a) da \quad (33)$$

$$= \frac{1}{K} \int_{a: T(s, a) = s_t} [F(G_{\phi^*}(s_t, a_k), a_k) - F(s, a)] da, \quad (34)$$

$s = g(s_t, a)$ 。需注意变量 $\{\Gamma_k\}_{k=1}^K$ 是独立的，并且 $\mathbb{E}[\Gamma_k] = 0, k = 1, \dots, K$ 根据

引理 2。因此，以下方程成立

$$\mathbb{P} \left(\left| \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(s_t, a_k) - \int_{a \in \mathcal{A}} F(s_t, a) da \right| \geq t \right) = \mathbb{P} \left(\left| \sum_{k=1}^K \Gamma_k \right| \geq t \right) \quad (35)$$

以及

$$\mathbb{P} \left(\left| \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(G_{\phi^*}(s_t, a_k), a_k) - \int_{a: T(s, a) = s_t} F(s, a) da \right| \geq t \right) = \mathbb{P} \left(\left| \sum_{k=1}^K \Lambda_k \right| \geq t \right). \quad (36)$$

由于 $F(s, a)$ 是一个 Lipschitz 函数，我们有

$$\begin{aligned} |\Gamma_k| &\leq \frac{1}{K} \int_{a \in \mathcal{A}} |F(s_t, a_k) - F(s_t, a)| da \\ &\leq \frac{L}{K} \int_{a \in \mathcal{A}} \|a_k - a\| da \leq \frac{L\mu(\mathcal{A})\text{diam}(\mathcal{A})}{K}. \end{aligned} \quad (37)$$

结合假设 3，即对于满足 $T(s, a) = s_t, a$ 的任意一对 (s, a) ，当固定 s 时，其唯一性成立，我们有

$$\begin{aligned} |\Lambda_k| &\leq \frac{1}{K} \int_{a: T(s, a) = s_t} |F(G_{\phi^*}(s_t, a_k), a_k) - F(s, a)| da \\ &\leq \frac{1}{K} \int_{a: T(s, a) = s_t} |F(G_{\phi^*}(s_t, a_k), a_k) - F(s, a_k) + F(s, a_k) - F(s, a)| da \\ &\leq \frac{1}{K} \int_{a: T(s, a) = s_t} L\|G_{\phi^*}(s_t, a_k) - s\| + L\|a_k - a\| da \\ &\leq \frac{L\mu(\mathcal{A})(\text{diam}(\mathcal{A}) + \text{diam}(\mathcal{S}))}{K}. \end{aligned} \quad (38)$$

引理 3 (霍夫丁不等式, Vershynin (2018))。设 $x_1, \dots, x_K$ 为独立随机变量。假设变量 $\{x_k\}_{k=1}^K$ 在区间 $[T_l, T_r]$ 内有界。那么对于任意 $t > 0$ ，我们有

$$\mathbb{P}\left(\left|\sum_{k=1}^K (x_k - \mathbb{E}x_k)\right| \geq t\right) \leq 2 \exp\left(-\frac{2t^2}{K(T_r - T_l)^2}\right). \quad (39)$$

将 $T_r = \frac{L}{K}\mu(\mathcal{A})\text{diam}(\mathcal{A})$ 及 $T_l = -\frac{L}{K}\mu(\mathcal{A})\text{diam}(\mathcal{A})$ 引理 3 与 (37) 结合，并将 $T_r = \frac{L}{K}\mu(\mathcal{A})(\text{diam}(\mathcal{A}) + \text{diam}(\mathcal{S}))$ 及 $T_l = -\frac{L}{K}\mu(\mathcal{A})(\text{diam}(\mathcal{A}) + \text{diam}(\mathcal{S}))$ 引理 3 与 (38) 结合，我们完成证明。 $\square$

B.3 引理 1 的证明

引理 1. 考虑一个连续任务 $(\mathcal{S}, \mathcal{A})$ ，其中我们定义了转移概率 (8) 和 (9)。定义 $\mathcal{T}_{s,f}$ 和 $\mathcal{T}_{0,s}$ 分别为从状态 s 开始并在状态 s_f 结束，以及从状态 s_0 开始并在状态 s 结束的轨迹集合。那么我们有

$$\forall s \in \mathcal{S} \setminus \{s_f\}, \int_{\tau \in \mathcal{T}_{s,f}} P_F(\tau) d\tau = 1 \quad (40)$$

$$\forall s \in \mathcal{S} \setminus \{s_0\}, \int_{\tau \in \mathcal{T}_{0,s}} P_B(\tau) d\tau = 1. \quad (41)$$

证明。我们通过强归纳法证明 (40) 成立，主要遵循 Bengio 等人 (2021b) 中引理 5 的证明过程，然后扩展到 (41) 是显而易见的。定义 d 为 $\mathcal{T}_{s,f}, s \neq s_f$ 中的最大轨迹长度，我们有：基本情况：如果 $d = 1$ ，则

$$\int_{\tau \in \mathcal{T}_{s,f}} P_F(\tau) d\tau = P_F(s \rightarrow s_f) = 1$$

成立。归纳步 $\mathcal{T}_{s,f} = \{(s \rightarrow s_f)\}$ 。骤：考虑 $d > 1$ ，注意到 (12)，我们有

$$\int_{\tau \in \mathcal{T}_{s,f}} P_F(\tau) d\tau = \int_{s' \in \mathcal{C}(s)} \int_{\tau \in \mathcal{T}_{s \rightarrow s',f}} P_F(\tau) d\tau ds' \quad (42)$$

$$= \int_{s' \in \mathcal{C}(s)} \int_{\tau \in \mathcal{T}_{s',f}} P_F(s'|s) P_F(\tau) d\tau ds' \quad (43)$$

$$= \int_{s' \in \mathcal{C}(s)} P_F(s'|s) ds' \int_{\tau \in \mathcal{T}_{s',f}} P_F(\tau) d\tau = 1, \quad (44)$$

其中最后一个等式由归纳假设得出。 $\square$

B.4 引理 2 的证明

引理 2. 设 $\{a_k\}_{k=1}^K$ 是从连续动作空间 $\mathcal{A}$ 中独立均匀采样的。假设 G_{ϕ^*} 能够最优地输出实际状态 s_t ，并且使用 (s_{t+1}, a_t) 。那么对于任意状态 $s_t \in \mathcal{S}$ ，我们有

$$\mathbb{E}\left[\frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(s_t, a_k)\right] = \int_{a \in \mathcal{A}} F(s_t, a) da \quad (45)$$

以及

$$\mathbb{E}\left[\frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(G_{\phi^*}(s_t, a_k), a_k)\right] = \int_{a: T(s, a) = s_t} F(s, a) ds, \quad (46)$$

其中 $s = g(s_t, a)$ 。

证明。由于 $\{a_k\}_{k=1}^K$ 是从连续动作空间 $\mathcal{A}$ 中独立且均匀地采样的，则我们有

$$\mathbb{E}[F(s_t, a_k)] = \frac{1}{\mu(\mathcal{A})} \int_{a \in \mathcal{A}} F(s_t, a) da. \quad (47)$$

因此，我们得到

$$\mathbb{E} \left[\frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(s_t, a_k) \right] = \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K \mathbb{E}[F(s_t, a_k)] \quad (48)$$

$$= \int_{a \in \mathcal{A}} F(s_t, a) da. \quad (49)$$

由于假设 3 成立，对于满足 $T(s, a) = s_t, a$ 的任意一对 (s, a) ，若固定 s ，则其唯一性成立，我们有

$$\mathbb{E}[F(G_{\phi^*}(s_t, a_k), a_k)] = \frac{1}{\mu(\mathcal{A})} \int_{a: T(s, a) = s_t} F(s, a) da,$$

其中 $s = g(s_t, a)$.

因此，我们得到

$$\begin{aligned} \mathbb{E} \left[\frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K F(G_{\phi^*}(s_t, a_k), a_k) \right] &= \frac{\mu(\mathcal{A})}{K} \sum_{k=1}^K \mathbb{E}[F(G_{\phi^*}(s_t, a_k), a_k)] \\ &= \int_{a: T(s, a) = s_t} F(s, a) da. \end{aligned}$$

于是证明完成。 $\square$

C CFLOWNETS 的伪代码

为了清晰起见，我们在算法 1 中展示了 CFlowNets 的伪代码。

Algorithm 1 Generative Continuous Flow Networks (CFlowNets) Algorithm

Initialize: Flow network θ ; a pretrained retrieval network G_ϕ ; and empty buffer $\mathcal{D}$ and $\mathcal{P}$

```

1: repeat
2:   Set  $t = 0, s = s_0$ 
3:   while  $s \neq \text{terminal}$  and  $t < T$  do
4:     Uniformly sample  $M$  actions  $\{a_i\}_{i=1}^M$  from action space  $\mathcal{A}$ 
5:     Compute edge flow  $F_\theta(s_t, a_i)$  for each  $a_i \in \{a_i\}_{i=1}^M$  to generate  $\mathcal{P}$ 
6:     Sample  $a_t \sim \mathcal{P}$  and execute  $a_t$  in the environment to obtain  $r_{t+1}$  and  $s_{t+1}$ 
7:      $t = t + 1$ 
8:   end while
9:   Store episodes  $\{(s_t, a_t, r_t, s_{t+1})\}_{t=1}^T$  in replay buffer  $\mathcal{D}$ 
10:  [Optional] Fine-tuning retrieval network  $G_\phi$  based on  $\mathcal{D}$ 
11:  Sample a random minibatch  $\mathcal{B}$  of episodes from  $\mathcal{D}$ 
12:  Uniformly sample  $K$  actions  $\{a_k\}_{k=1}^K$  from action space  $\mathcal{A}$  for each state in  $\mathcal{B}$ 
13:  Compute parent states according to  $\{G_\phi(s, a_k)\}_{k=1}^K$  for each state in  $\mathcal{B}$ 
14:  Inflows:
15:     $\log[\epsilon + \sum_{k=1}^K \exp F_\theta^{\log}(G_\phi(s_t, a_k), a_k)]$ 
16:  Outflows or reward:
17:     $\log[\epsilon + \lambda R(s_t) + \sum_{k=1}^K \exp F_\theta^{\log}(s_t, a_k)]$ 
18:  Update flow network  $F_\theta$  according to (20)
19: until convergence

```

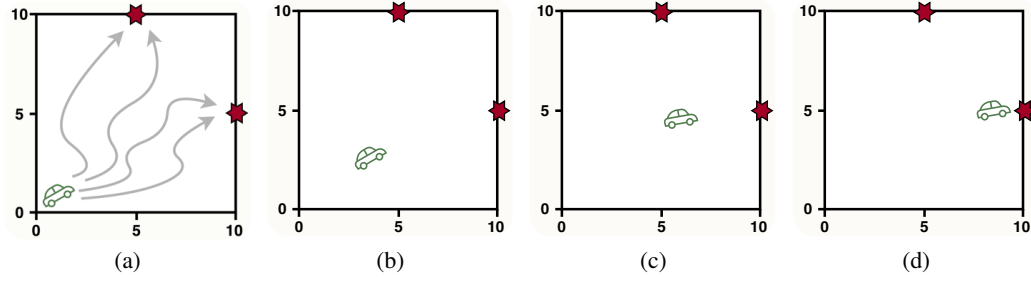


图 7: Point-Robot-Sparse 任务的可视化。

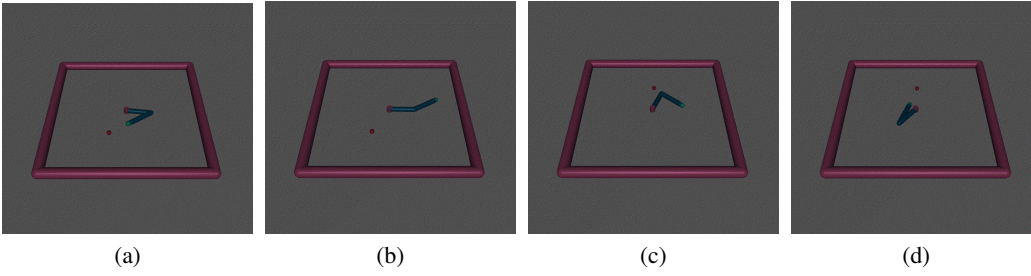


图 8: Reacher-Goal-Sparse 任务的可视化。

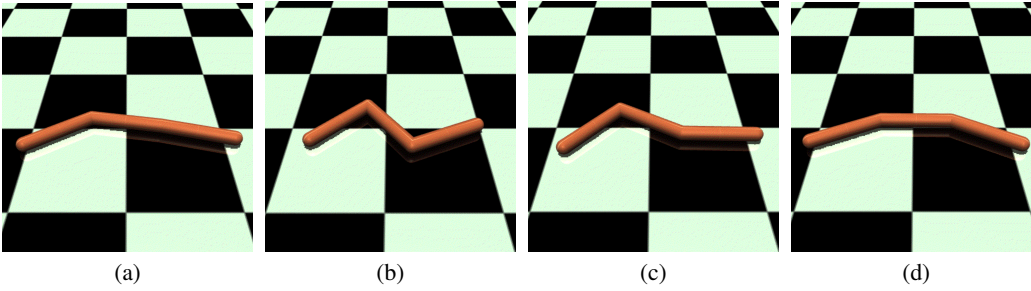


图 9: Swimmer-Sparse 任务的可视化。

附录 D 额外实验

附录 D.1 环境的可视化

如图 7、8 和 9 所示，我们提供了 Point-Robot-Sparse、Reacher-Goal-Sparse 和 Swimmer-Sparse 任务的可视化。在 Point-Robot-Sparse 中，代理的目标是在两个不同的目标之间导航。代理从起始坐标 $(0,0)$ 开始，每次移动一步向目标坐标前进。环境有两个目标坐标 $(5,10)$ 和 $(10,5)$ ，最大回合长度为 12，仅当最后一步到达时环境才返回奖励。奖励通过测量代理当前位置与目标节点之间的距离来发放，距离越近，奖励越大。每次代理可以从任意角度向上右方向迈出一大步。

Reacher-Goal-Sparse 和 Swimmer-Sparse 都改编自 OpenAI Gym 的 MuJoCo 环境。在 Reacher-Goal-Sparse 中，“Reacher”是一个两关节的机械臂。代理的目标是通过移动机器人的末端执行器来达到随机生成的目标。图 8 显示了机械臂的运动过程。通过调整铰链关节处施加的扭矩，末端执行器可以逐渐接近目标。在 Swimmer-Sparse 中，“swimmer”悬停在一个

二维池塘，目标是以尽可能快的速度向左或右移动。图 9 展示了机器人在运动过程中的形状变化。通过施加扭矩于旋翼并利用流体摩擦，机器人可以游得更快。我们将这两个环境的最大步数设置为 50。对于 Reacher-Goal-Sparse，当到达最后一步时，环境会返回一个奖励，该奖励衡量代理与随机生成的目标之间的距离。代理越接近目标，奖励越大。对于 Swimmer-Sparse，从起点向左或向右移动的距离越远，返回的奖励越大。

D.2 额外分析

图 10 显示了不同算法在 Point-Robot-OneGoal-Sparse 任务上的平均奖励和奖励分布情况，在此任务中，代理需要导航到特定位置。图 10

(a) 表明 CFlowNets 可以获得与其他基于 RL 的算法相比最高的平均回报。在图 10(b) 中，所有算法都能很好地拟合一目标设置下的奖励分布，而 CFlowNets 表现更好。请注意，由于在这种单一目标情况下最大化奖励是最优策略，且策略不难学习，因此 RL 算法也可以学习这种奖励分布。

在图 11 中，我们提供了不同算法在 Point-Robot-Sparse 任务上，分别针对点 (4,8)、点 (8,4) 和点 (7,7) 的行动奖励分布，总时间步为 $2e4$ 。请注意，与图 2 不同，其中总时间步为 $1e5$ ，这里我们展示的是 $2e4$ 总时间步的结果，因为在该任务中发现 DDPG 在 $1e5$ 时间步后过拟合。因此，我们展示了没有过拟合的结果，以便进行更公平的比较。可以看到，无论在哪一点，CFlowNets 的策略都能更好地匹配实际奖励分布。例如，在点 (4,8) 和 (8,4)，CFlowNet 倾向于选择引导代理分别向 (5,10) 和 (10,5) 移动的动作。对于位于两个目标之间的位置（点 (7,7)），有两个方向可以让代理以高奖励到达目标。相比之下，由 RL 算法学习的策略只能偶尔-

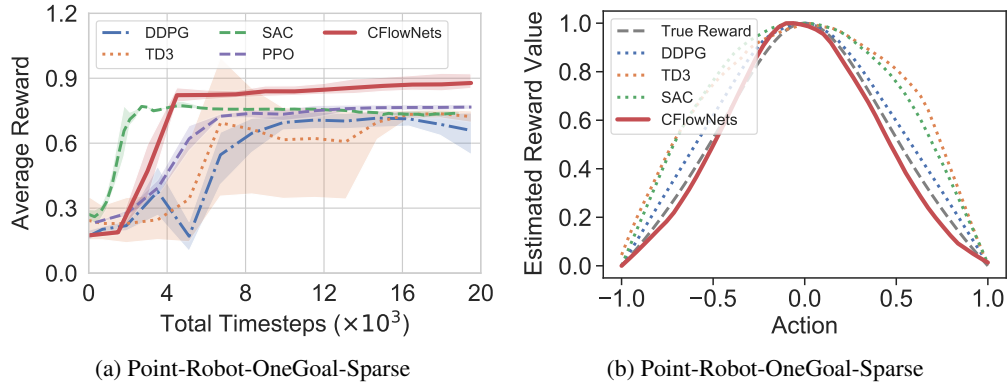


图 10: CFlowNets、DDPG、TD3、SAC 和 PPO 在 Point-Robot-OneGoal-Sparse 任务上的平均奖励和奖励分布。

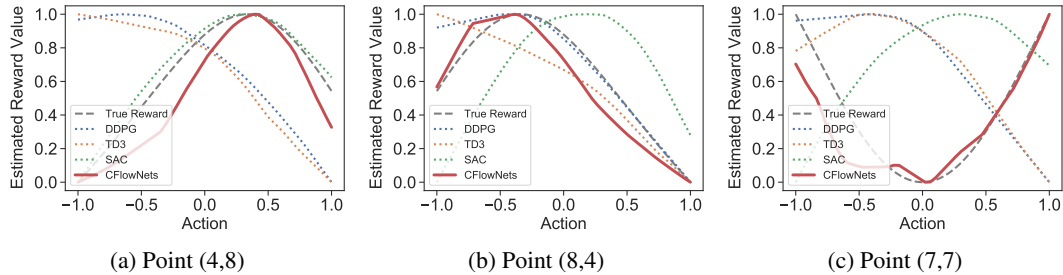


图 11: Point-Robot-Sparse 任务上不同点的奖励分布。

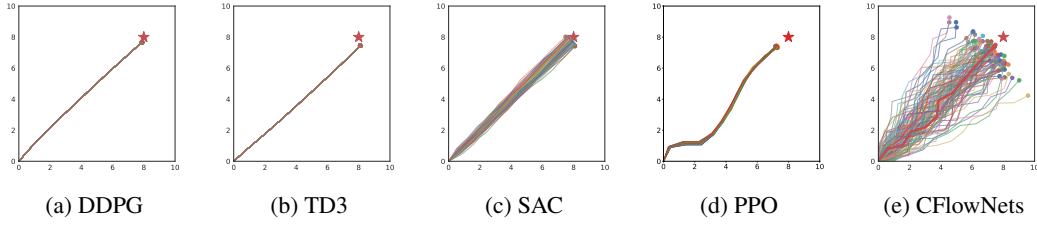


图 12: Point-Robot-OneGoal-Sparse 任务上的采样轨迹。

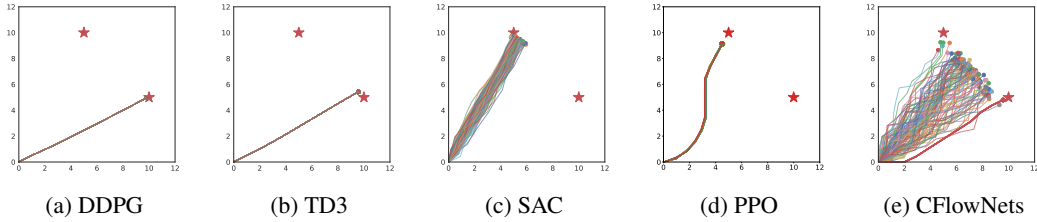


图 13: Point-Robot-Sparse 任务上的采样轨迹。

这些策略仅与某一特定点的真实奖励分布相匹配，并不能稳定地匹配每一个点。这也表明通过强化学习算法学习到的策略相对简单。CFlowNets 学习到了更多样化的策略，使代理能够以高奖励到达不同的目标，而其他方法通常只会找到一个目标而不是所有潜在的高奖励位置。

图 12 和图 13 展示了不同算法生成的轨迹可视化结果。在 Point-Robot-OneGoal-Sparse 任务中，DDPG、TD3 和 PPO 的轨迹是单一的，而 SAC 可以从策略概率分布中选择动作，因此可以获得不同的轨迹。相比之下，CFlowNets 找到了更多样化的轨迹，并且也找到了最高奖励的目标（加粗红色轨迹），这意味着 CFlowNets 可以更好地探索目标附近的区域。在 Point-Robot-Sparse 任务中，基于强化学习的算法只寻求一个目标。然而，CFlowNets 可以找到所有目标。

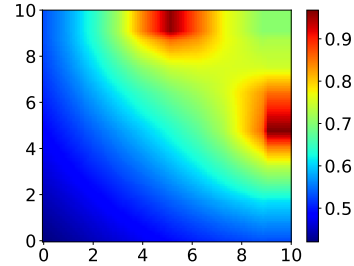


图 14: Point-Robot-Sparse 任务上的奖励分布。

值得注意的是，在图 13(e) 中，CFlowNets 采样的轨迹密度并不像

图 12(e) 在最大奖励附近那样密集。相反，在对角线上更密集。这是因为大多数位置选择向上和向右行动的概率相对较高，因此更容易组合成对角线方向。此外，两个目标之间的线上的奖励也不小。当根据流模型的输出作为概率进行采样时，许多轨迹本身更有可能到达对角线。图 14 显示了 Point-Robot-Sparse 的真实奖励分布，其中接近两个目标和两个目标之间线上的奖励更高。

D.3 高速公路停车稀疏实验结果

我们在 Highway-Parking-Sparse 任务上评估 CFlowNets 的性能，这是一个自我车辆控制任务。如图 15 所示，目标是通过调整控制器使自我车辆以适当的方向停入指定空间。车辆观测维度为 18，包括车辆与停车位之间的距离、车辆速度、三角航向信息、代理应尝试实现的目标以及当前目标

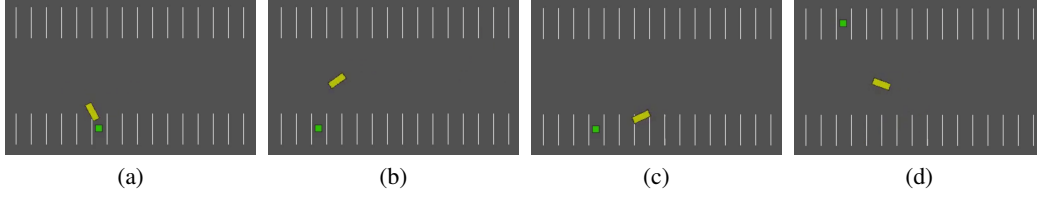


图 15: Highway-Parking-Sparse 任务的可视化。

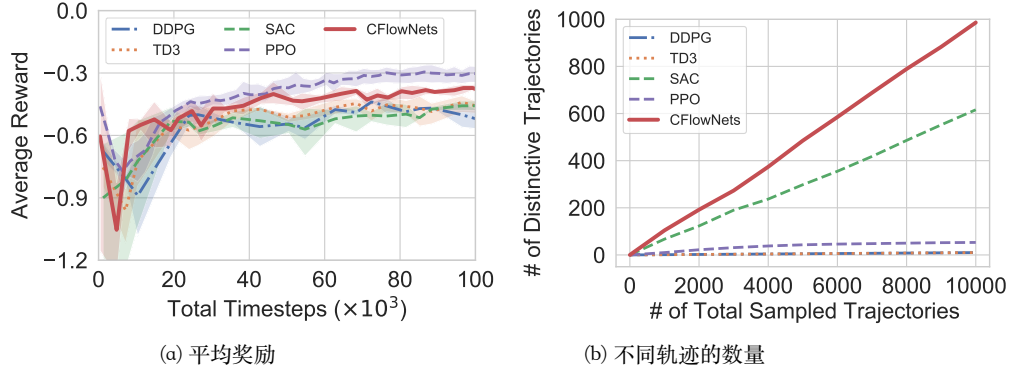


图 16: 在 Highway-Parking-Sparse 任务上, CFlowNets、DDPG、TD3 和 SAC 经过 10000 次探索后生成的平均奖励和有效不同轨迹数量。

所达成的。动作空间包括对油门和转向角度的控制, 奖励函数设置为自我车辆与停车位之间的距离。图 16 展示了随着训练进程的不同算法探索的平均奖励和有效不同轨迹数量, 这表明 CFlowNets 的性能比其他基于 RL 的算法更有前景。即使对于更高维度的连续任务, CFlowNets 也具有非常有竞争力的奖励结果 (优于 DDPG、TD3 和 SAC), 同时在探索性能方面远超基于 RL 的算法。

D.4 基线

我们将提出的 CFlowNets 与以下基线进行比较:

- 深度确定性策略梯度 (DDPG) (Lillicrap 等人, 2015) 。 <https://github.com/sfujim/TD3/blob/master/DDPG.py>
- 双延迟深度确定性策略梯度 (TD3) (Fujimoto 等人, 2018) 。 <https://github.com/sfujim/TD3>
- 软演员 - 评论家 (SAC) (Haarnoja 等人, 2018b) 。 https://github.com/denisyarats/pytorch_sac/
- 近端策略优化 (PPO) (Schulman 等人, 2017) 。 https://github.com/DLR-RM/stable-baselines3/blob/master/stable_baselines3/ppo/ppo.py

D.5 超参数

表 1、表 2、表 3、表 4 和表 5 提供了不同环境中所有比较方法的超参数。

对于“总时间步”、“开始训练时间步”、“最大剧集长度”、“演员网络隐藏层”、“评论家网络隐藏层”、“优化器”、“学习率”和“折扣因子”, 我们将它们设置为所有算法相同, 以实现公平比较。对于基线算法的具体参数, 我们保持与原始代码相同, 以获得良好的性能。对于我们的 CFlowNets 的具体参数, 我们将样本流的数量设置为 100, 动作概率缓冲区大小设置为 1000, 以平衡性能和计算负载。需要注意的是, CFlowNets 不需要像其他强化学习算法那样大的回放缓冲区, 因为 CFlowNets 的探索能力优于其他算法。并且可以从较小的回放缓冲区中学到一个好的策略。这也是 CFlowNets 相对于基于强化学习的算法的优势之一。

表 1: 不同环境下的 CFlowNets 超参数。

	Point-Robot-Sparse	Reacher-Goal-Sparse	Swimmer-Sparse
Total Timesteps	100,000	100,000	100,000
Start Training Timestep	4,000	7,500	7,500
Max Episode Length	12	50	50
Flow Network Hidden Layers	[256,256]	[256,256]	[256,256]
Retrieval Network Hidden Layers	[256,256,256]	[256,256,256]	[256,256,256]
Optimizer	Adam	Adam	Adam
Learning Rate	0.0003	0.0003	0.0003
Batchsize	128	128	128
Number of Sample Flows	100	100	100
Action Probability Buffer Size	1,000	1,000	10,000
Replay Buffer Size	8,000	2,000	2,000
ϵ	1.0	1.0	1.0

表 2: 不同环境下的 DDPG 超参数。

	Point-Robot-Sparse	Reacher-Goal-Sparse	Swimmer-Sparse
Total Timesteps	100,000	100,000	100,000
Start Training Timestep	4,000	7,500	7,500
Max Episode Length	12	50	50
Actor Network Hidden Layers	[256,256]	[256,256]	[256,256]
Critic Network Hidden Layers	[256,256]	[256,256]	[256,256]
Optimizer	Adam	Adam	Adam
Learning Rate	0.0003	0.0003	0.0003
Batchsize	256	256	256
Discount Factor	0.99	0.99	0.99
Replay Buffer Size	100,000	100,000	100,000
Target Network Update Rate	0.005	0.005	0.005

表 3: 不同环境下的 TD3 超参数。

	Point-Robot-Sparse	Reacher-Goal-Sparse	Swimmer-Sparse
Total Timesteps	100,000	100,000	100,000
Start Training Timestep	4,000	7,500	7,500
Max Episode Length	12	50	50
Actor Network Hidden Layers	[256,256]	[256,256]	[256,256]
Critic Network Hidden Layers	[256,256]	[256,256]	[256,256]
Optimizer	Adam	Adam	Adam
Learning Rate	0.0003	0.0003	0.0003
Batchsize	128	128	128
Discount Factor	0.99	0.99	0.99
Replay Buffer Size	100,000	100,000	100,000
Gaussian Exploration Noise	0.1	0.1	0.1
Target Network Update Rate	0.005	0.005	0.005

表 4: 不同环境下的 SAC 超参数。

	Point-Robot-Sparse	Reacher-Goal-Sparse	Swimmer-Sparse
Total Timesteps	100,000	100,000	100,000
Start Training Timestep	4,000	7,500	7,500
Max Episode Length	12	50	50
Actor Network Hidden Layers	[256,256]	[256,256]	[256,256]
Critic Network Hidden Layers	[256,256]	[256,256]	[256,256]
Optimizer	Adam	Adam	Adam
Learning Rate	0.0003	0.0003	0.0003
Batchsize	1024	1024	1024
Discount Factor	0.99	0.99	0.99
Replay Buffer Size	100,000	100,000	100,000
Target Update Interval	1	1	1

表 5: 不同环境下的 PPO 超参数。

	Point-Robot-Sparse	Reacher-Goal-Sparse	Swimmer-Sparse
Total Timesteps	100,000	100,000	100,000
Max Episode Length	12	50	50
Policy Network Hidden Layers	[64,64]	[64,64]	[64,64]
Value Network Hidden Layers	[64,64]	[64,64]	[64,64]
Optimizer	Adam	Adam	Adam
Learning Rate	0.0003	0.0003	0.0003
Batchsize	64	64	64
Discount Factor	0.99	0.99	0.99
GAE Parameter	0.95	0.95	0.95
Timesteps per Update	2048	2048	2048
Number of Epochs	10	10	10
Clipping Parameter	0.2	0.2	0.2
Value Loss Coefficient	0.5	0.5	0.5